Language-3D: End-to-End Generation of Manufacturing-Ready Robot Assemblies from Natural Language

Youcheng Xiong*
*Future Laboratory, Tsinghua University
Beijing, China
†University of Macau
Macau SAR, China
dc42857@um.edu.mo

***Abstract*—We present Language-3D, a multi-agent system that converts natural language descriptions into *engineering packages* for robot assemblies—including STL/STEP meshes, URDF models, bills of materials (BOM), firmware templates, and ROS2 package skeletons. Unlike prior work that generates either single CAD parts [1], [2], voxel-based block assemblies [3], or URDF-only morphologies [4], Language-3D produces the full set of artifacts needed for manufacturing and deployment. STL meshes are validated for watertightness, URDF models are validated in MuJoCo physics simulation, and parts use real COTS fastener specifications. The system is validated in simulation only; physical 3D printing and hardware deployment remain as future work.**

The system features: (1) a COTS-driven knowledge base of 94 part templates (56 real commercial components with verified ISO/DIN specifications), (2) eight connection types with real CAD feature generation (bolted clearance holes, press-fit interference bores, snap-fit joints, etc.), (3) a dual-channel verification architecture combining vision-language model (VLM) inspection with geometric arbitration (FCL collision sweep, assembly sequence validation), (4) a MuJoCo physics validation pipeline with native actuator models verifying that generated robots can drive, articulate, and grasp, and (5) a case-based experience store that retrieves verified-good past cases to prime new generations (lexical retrieval; successes only).

We evaluate on seven robot configurations (2–7-DOF arms and a 4-wheel dual-arm mobile robot) with a composite quality score $Q \in [0, 1]$ (a relative rank, not an absolute quality percentage), achieving a mean of 0.68 (range 0.49–0.85) with 7/7 distinct values. The composite's grasp term is a per-case success *rate* (not best-of-N), exposing that the 6-DOF arm grasps in only 12% of runs (1 of 8) and the 7-DOF in 0%. We are transparent that no external benchmark exists for the NL→robot-package task and that the static-grasp, lift, and per-joint collision checks have known scope limits (§IV-F).

***Index Terms*—Natural language to CAD, robot assembly generation, URDF, multi-agent systems, vision-language models, MuJoCo simulation, generative design**

I. Introduction

Automated robot design from natural language is a long-standing goal at the intersection of AI, CAD, and robotics. Recent advances in large language models (LLMs) and vision-language models (VLMs) have enabled systems that translate textual descriptions into 3D geometry, articulated assemblies, or robot morphologies. However, existing work covers only fragments of the full design-to-manufacture pipeline:

NL → single CAD part: CAD-Llama [1] generates parametric construction sequences, and STEP-LLM [2] produces STEP-format models. Both focus on single parts without assembly structure.

NL → CAD assembly: ArtiCAD [5] is the first training-free multi-agent system for articulated CAD assemblies, outputting editable FreeCAD code and URDF files. However, it does not produce manufacturing-grade meshes (STL/STEP), bills of materials, firmware, or ROS2 packages, nor does it include physical simulation validation or real COTS part specifications.

NL → robot design: Blox-Net [3] combines VLM supervision with physics simulation to generate block-based assemblies, but is limited to voxel primitives and requires a physical robot for iterative reset. Text2Robot [6] (ICRA 2025) generates quadruped robots from text via 3D mesh + URDF + RL gait evolution + 3D printing with real servos—overlapping Language-3D on NL/URDF/sim, but limited to walking robots (no arms, grippers, or BOM/firmware), and has physical hardware validation we lack. RoboMorph [4] uses LLMs to evolve modular robot URDFs, but produces skeleton configurations without CAD geometry, assembly connections, or manufacturing artifacts.

The gap: No existing system produces a *complete engineering package*—the full set of artifacts (STL, STEP, URDF, BOM, firmware, assembly guide, ROS2 package) needed to build and deploy a robot. We define this as the **NL → Robot Assembly Engineering Package** task, and note that physical manufacturing verification (3D printing, Gazebo deployment, hardware testing) is an orthogonal validation step that complements but does not replace artifact generation.

Contributions. This paper makes the following contributions:

1) **Task formulation**: We formalize NL → manufacturing-ready robot assembly package as a new task with evaluation criteria spanning geometry, physics, and manufacturing.

2) **System**: Language-3D, a multi-agent pipeline producing STL/STEP/URDF/BOM/firmware/ROS2 package structures from NL, with 8 real connection types and a 56-part COTS knowledge base.
3) **Dual-channel verification**: VLM visual inspection combined with geometric arbitration (FCL collision sweep + assembly sequence validation) that can override VLM false-negatives.
4) **Evaluation**: Seven benchmark cases evaluated by a composite quality score (gate + geometric mean of robustness/functionality/reliability; $Q \in [0, 1]$ relative rank, mean 0.68, 7/7 distinct values), with all cases passing MuJoCo physics stability. An eighth humanoid topology was also generated to probe generalization beyond fixed-base arms but is reported qualitatively, not in the scored benchmark (see §V-B).
5) **Open-source release**: The full pipeline, knowledge base, evaluation harness, and benchmark run archive are released under an MIT license[1]. To our knowledge this is the first open-source system to produce the complete manufacturing-ready package—STL + STEP + URDF + BOM + firmware + ROS2 package + assembly guide—from natural language. Prior open-source NL→robot/assembly systems produce a strict subset of these artifacts: Text2Robot [6] (mesh + URDF + assembly PDF), CADSmith [7] (single-part STEP/STL), and Articraft [8] (URDF + STL for general articulated objects, without BOM/firmware/ROS2/COTS specs). Our differentiation is the *robot engineering package*—combining the COTS knowledge base, eight real CAD connection types, dual-channel verification, and MuJoCo physics validation.

II. Related Work

A. NL-Driven CAD Generation

CAD-Llama [1] (CVPR 2025) uses LLMs to generate parametric CAD construction sequences for single parts. STEP-LLM [2] extends this to industry-standard STEP format. LLM4CAD [9] explores multimodal approaches. CADCode-Verify [10] (ICLR 2025) introduces a benchmark with vision-language verification. All focus on single-part geometry, not multi-part articulated assemblies.

B. NL-Driven Assembly and Robot Design

ArtiCAD [5] is the closest prior work: a training-free multi-agent system generating editable, articulated CAD assemblies via code generation. It exports URDF files (confirmed in [5], §6) but its pipeline outputs are editable FreeCAD code (parametric B-rep), not directly-exported STL/STEP meshes, BOMs, firmware, or ROS2 packages, and it lacks physical simulation validation. CADSmith [7] (CMU, 2026) shares our multi-agent architecture (Planner→Coder→Executor→Validator→Refiner) for NL→CadQuery code generation with programmatic geometric validation, but focuses on single parts rather than articulated assemblies. Concurrently, Embodied CAD [11] argues that LLM-generated CAD code requires solver-grounded reasoning for industrial reliability—a principle aligned with our deterministic Solver stage. Blox-Net [3] (CoRL 2024 Workshop) combines VLM supervision with physics simulation for generative design-for-robot-assembly, but is limited to voxel blocks. RoboMorph [4] evolves modular robot URDFs via LLM, producing morphology configurations without CAD geometry or assembly connections. RoboGen [12] (ICML 2024) focuses on automated skill learning via generative simulation, not robot body generation. URDF-Anything [13] (NeurIPS 2025) is the closest end-to-end URDF generator, using a 3D multimodal LLM to reconstruct articulated objects from point-cloud + text input into executable URDF; however, it requires visual observation of an existing object and does not produce STL meshes, BOMs, or firmware. AutoURDF [14] (CVPR 2025) reconstructs robot URDFs unsupervised from point-cloud time-series (no language input). RobotDesignGPT [15] generates robot URDFs from text+image via a VLM, but like URDF-Anything produces morphology without CAD geometry, BOM, or a manufacturing package. Concurrently, Articraft [8] (Cambridge/Oxford, 2026) reduces articulated 3D asset generation to code generation against a domain-specific SDK, producing simulation-ready URDF + STL models at scale (Articraft-10K: 10K assets across 245 categories) and released as open source. It targets general articulated objects (furniture, household items) rather than robot engineering packages, and does not produce BOMs, firmware, ROS2 packages, assembly guides, or COTS part specifications. Self-Improving CAD Agents [16] introduces an LLM CAD agent with a finite-element-analysis (FEA) feedback loop (CadQuery + CalculiX) for self-improvement—a capability on our roadmap (FEA structural analysis, currently unimplemented). Text-to-Robotic-Assembly [17] (MIT, 2025) generates multi-component physical assemblies from text executed by a robot arm ("robot make me a chair"), overlapping on assembly + physical execution but targeting furniture assembly rather than robot-body packages. VLMgineer [18] (ICLR 2026, UPenn) co-designs manufacturable robotic tools via VLM + evolutionary search, sharing the VLM-loop + manufacturability theme but focusing on tool design, not full robot engineering packages. Scenethesis [19] (ICLR 2026, NVIDIA) generates physically plausible 3D interactive scenes from text, addressing physical validity at the scene level rather than robot-assembly level.

Table I summarizes the comparison.

III. Method

A. System Architecture

The core design principle is *determinism where it matters, intelligence where it helps*: the LLM handles open-ended generation (assembly topology, part selection) while deterministic stages (Solver, CAD, sanitizers, physics) handle geometry and validation, ensuring that output is reproducible and physically

[1]Code and benchmark archive: [anonymizedfordouble-blindreview]; the camera-ready version will include the permanent repository URL.

TABLE I
COMPARISON OF LANGUAGE-3D WITH STATE-OF-THE-ART SYSTEMS FOR NATURAL-LANGUAGE-DRIVEN ROBOT/ASSEMBLY GENERATION. ALL CLAIMS VERIFIED AGAINST $\geq$2 INDEPENDENT SOURCES.

System	NL	Assembly	CAD Geometry			Manufacturing		Validation	
			Param.	STL	STEP	BOM	Firmware	Physics	Dual-VLM
CAD-Llama[1]	✓	–	✓	–	–	–	–	–	–
STEP-LLM[2]	✓	–	–	–	✓	–	–	–	–
LLM4CAD[9]	✓	–	✓	–	–	–	–	–	–
ArtiCAD[5]	✓	✓	✓	–*	–	–	–	–†	✓
Articraft[8]	✓	✓	–	✓	–	–	–	–	–
Blox-Net[3]	✓	✓	–	voxel	–	–	–	✓	✓
RoboMorph[4]	✓	✓	–	–	–	–	–	✓	–
Language-3D (ours)	✓	✓	✓	✓	✓	✓	✓	✓	✓

*ArtiCAD generates editable FreeCAD code from which STL may be derivable, but STL mesh export is not reported in the paper. Sources: arXiv HTML §6 + Bytez summary (both confirm URDF export; neither mentions STL/BOM/firmware).

Articraft (arXiv:2605.15187, Cambridge/Oxford, open source) generates articulated 3D assets as URDF + STL via an LLM coding agent writing against a domain SDK. It targets general articulated objects (furniture, household items; Articraft-10K dataset: 10K assets, 245 categories), not robot engineering packages—no BOM, firmware, ROS2 structure, assembly guides, or COTS part specs. Sources: arXiv abstract + HTML + GitHub (mattzh72/articraft).

†ArtiCAD mentions "physical prototyping" in its abstract, but does not describe automated physics simulation validation (e.g., contact, grasp, stability tests). We mark this as "–" to indicate no reported physics simulation pipeline; the distinction between "physical prototyping" (manual fabrication) and "physics simulation validation" (automated) should be clarified in future revisions.

Experience store: ArtiCAD additionally ships a self-evolving experience store using FAISS partitioning into Good/Issue buckets (arXiv:2604.10992v2, Review Agent section). Language-3D implements an analogous retrieve-before/store-after mechanism, but with lexical retrieval (keyword/category/DOF scoring) rather than embeddings, and stores only verified-good cases. This column is omitted from the table because only ArtiCAD and Language-3D have such a mechanism, and the retrieval backends are not directly comparable.

Key: NL = natural language input; Assembly = multi-part articulated assembly (not single part); Param./STL/STEP = CAD geometry output formats; BOM = bill of materials with real part numbers; Firmware = embedded code (servo/IK/motor drivers); Physics = simulation-based validation; Dual-VLM = dual-channel verification (VLM + geometric arbitration).

grounded. This separation also bounds the variance: a failed LLM generation can be re-run without re-deriving physics or connection geometry.

Figure 1 shows the pipeline. Natural language input is processed by a multi-agent chain: **Architect** (GLM-5.2 generates assembly JSON, primed by retrieved past cases), **Solver** (computes 3D positions via anchor constraints + FCL collision-aware range clamping), **CAD Engineer** (FreeCAD 1.1 generates per-part STL/STEP meshes with connection features), and a dual-channel **Verifier** (GLM-4.6V + geometric arbitration). A **Fixer** routes failures back to the appropriate upstream stage by failure type. Verified assemblies are recorded into a case-based experience store for future retrieval and exported as complete engineering packages.

B. COTS-Driven Knowledge Base

The knowledge base contains 94 part templates, of which 56 are real commercial components (`real_part=True`) with verified specifications: TowerPro MG996R servos (40.7×19.7×42.9 mm), JX DS3218 (20 kg-cm), SG90 micro servos, NEMA17/23 steppers, ISO/DIN fasteners (M3–M8 bolts, nuts, washers with verified head diameters, pitches, and clearance hole specs), and miniature bearings (608, 623, 625). Arm link lengths and base dimensions are profile-scaled (desktop vs. mobile) but servos are always sourced from the catalog—never invented or rescaled.

C. Connection Engine

Eight connection types generate real CAD features via FreeCAD operation dictionaries:

- **Bolted**: ISO 273 clearance holes + DIN 912 counterbores, with shared bolt patterns ensuring alignment across mating parts.
- **Press-fit**: H7/p6 interference bore with shoulder (ISO 286 tolerance).
- **Snap-fit**: Cantilever hooks with undercuts.
- **Adhesive**: Bonding grooves with controlled gap.
- **Welded**: V-groove bevel preparation (butt) or no-prep (fillet).
- **Magnetic**, **Dowel-pin** (H7 slip-fit), **Set-screw** (radial threaded hole).

Connection selection is physics-aware: when the LLM omits a connection method, a sanitizer assigns one by part category (e.g., bearings → press-fit, never bolted; sensors and foot pads → adhesive bonding; servos → M2 threaded holes). When the LLM specifies a method explicitly, it is respected. All eight types have functional geometry generators, though in practice bolted and press-fit dominate because most robot joints are structural or bearing-mounted and the LLM tends to default to bolted.

D. Dual-Channel Verification

Channel 1 (VLM): GLM-4.6V inspects panoramic renders (iso/front/top/right) for structural integrity, plus a dedicated gripper close-up for finger geometry.

Channel 2 (Geometric Arbitration): A deterministic geometric oracle runs FCL mesh-based collision detection (7 samples per joint across its range), assembly-sequence validation (parent-before-child tree feasibility), and center-of-mass stabil-

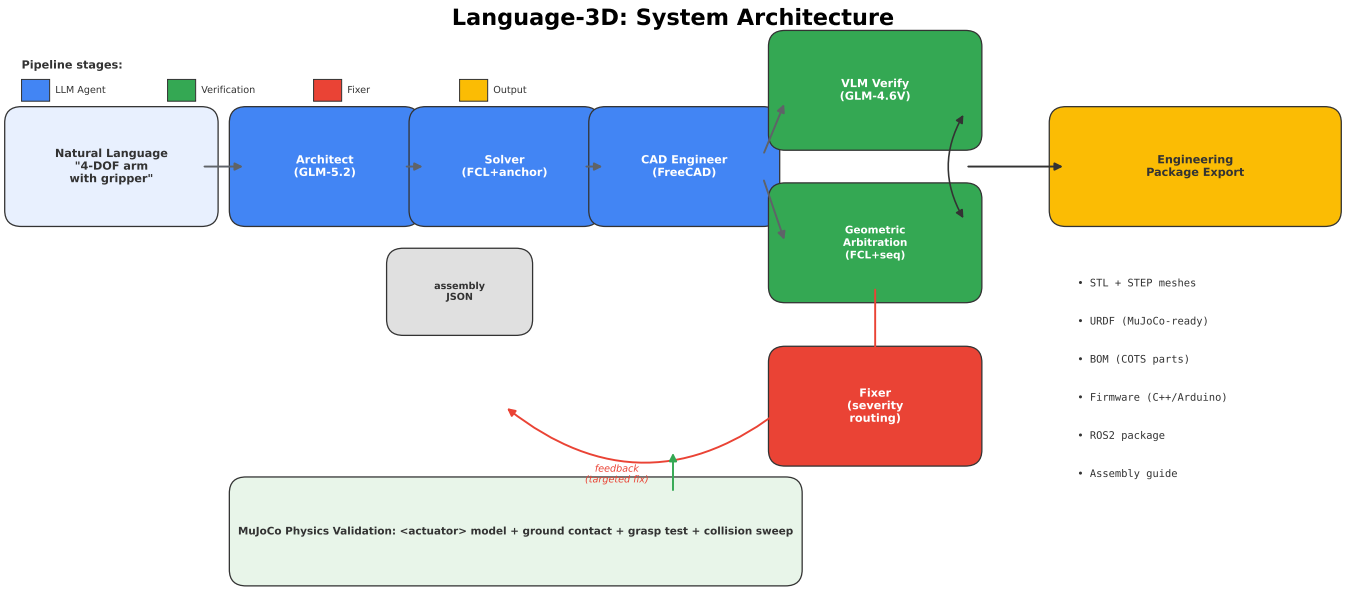

Fig. 1. Language-3D system architecture. NL input flows through the multi-agent pipeline (Architect → Solver → CAD Engineer), verified by dual channels (VLM + geometric arbitration), with a Fixer feedback loop. Output is a complete engineering package validated in MuJoCo physics simulation.

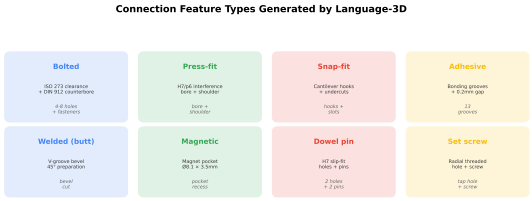

Fig. 2. Eight connection types with real CAD features. Each generates geometry-specific boolean operations (clearance holes, interference bores, snap hooks, etc.).

ity (support polygon check). The geometric channel overrides the VLM in two cases: (1) *false-alarm filter*—when the VLM reports a defect that geometry proves absent (e.g., "wheels above ground" on grounded wheels), the complaint is removed; (2) *assembly-sequence enforcement*—when geometry violates parent-before-child tree feasibility, the result is forced to FAIL regardless of VLM verdict. The collision-range sweep is advisory (narrows joint ranges but does not veto). Center-of-mass and proportion checks inform the Fixer's routing but do not directly override the VLM. In our historical passing runs, the false-alarm override triggers infrequently (a single-digit percentage of cases at most, as an upper bound that includes Fixer-resolved rounds); the assembly-sequence enforcement is rarer still. The override only removes geometrically-refutable complaints—hard structural defects (mesh collision, disconnected tree) always force FAIL and cannot be overridden.

E. Case-Based Experience Store

Following ArtiCAD's [5] observation that accumulated design knowledge improves future generations, Language-3D maintains an experience store of past verified-good assemblies cases. The store implements a *retrieve-before / store-after* pattern: before the Architect generates a new assembly, the store is queried for similar past cases (matched by robot category, keyword overlap, and DOF proximity), and the retrieved cases are injected as additional few-shot precedent; after the pipeline verifies an assembly, its distilled record (DOF, joint-type histogram, converged default angles) is appended for future retrieval. Retrieval is lexical (weighted scoring mirroring the existing template-search API), not embedding-based, because the project's LLM backend exposes no embedding endpoint—this is less semantically expressive than ArtiCAD's FAISS partitioning but runs with no external dependency. The store is successes-only (failed assemblies are never stored) and prunes least-retrieved entries past a per-category cap. On a fresh checkout the store is empty, so generation falls back to the parametric examples and the prompt is byte-identical to the pre-store baseline.

F. MuJoCo Physics Validation

The system injects a native MuJoCo `<actuator>` model (replacing hand-rolled PD control): `<position>` actuators with kp=50/kv=5 and `forcerange`=±5 N·m for arm joints, `<motor>` actuators for wheels. A ground plane and stiff contacts (`solref`=0.005, `solimp`=0.99) are injected via MJCF patching. Three physics tests verify the generated robot: (1) PD-hold stability (joint angle error $< 1°$), (2) joint actuation (≥ 4 actuated DOFs), and (3) three-phase grasp (zero-G close → gravity hold → lift). Figure 3 shows example generated robots in both render and simulation.

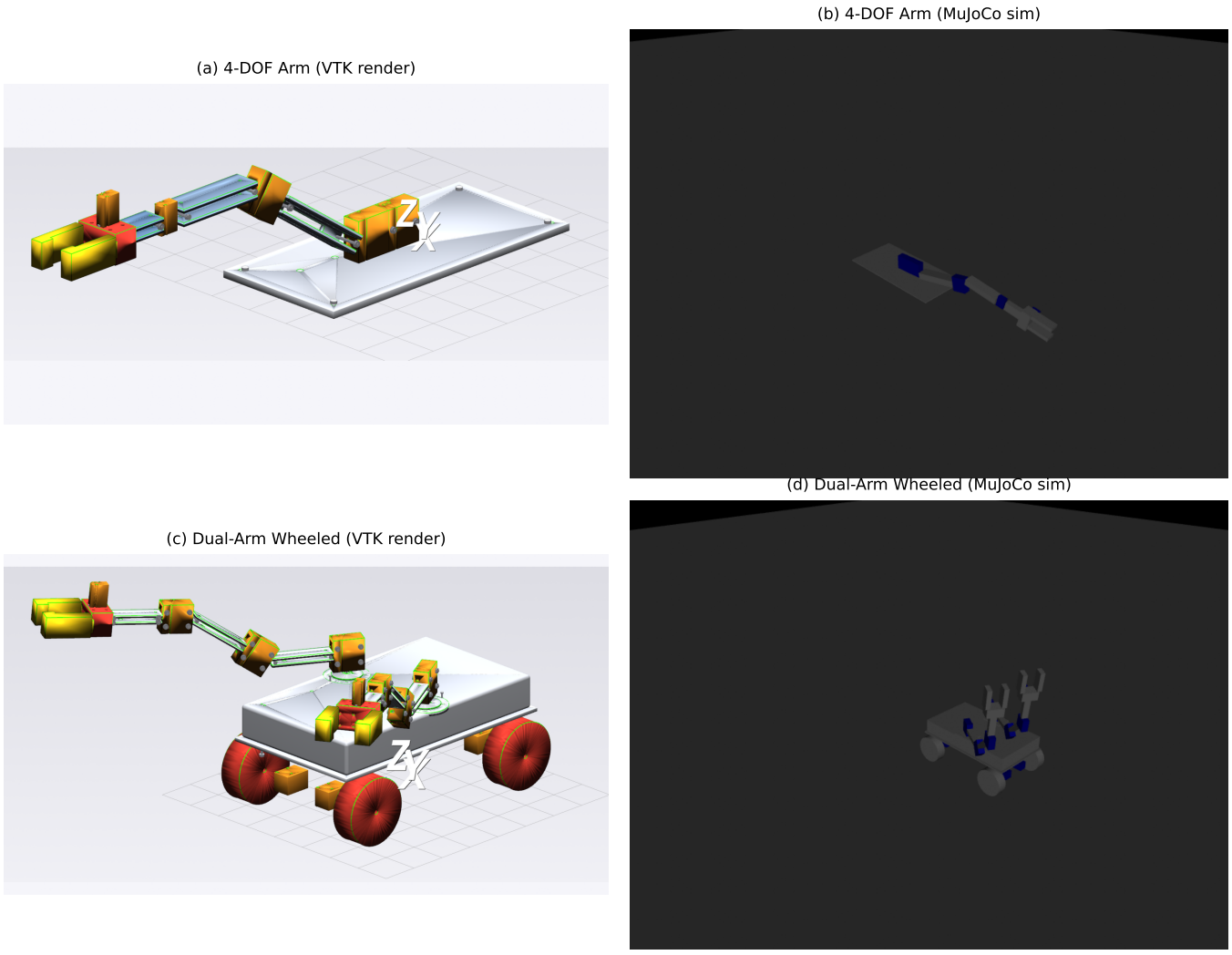

Fig. 3. Generated robots. (a) 4-DOF arm render, (b) 4-DOF arm in MuJoCo simulation performing a reach gesture, (c) 4-wheel dual-arm robot render, (d) dual-arm robot driving in MuJoCo simulation.

IV. Evaluation

A. Benchmark Cases

We evaluate on seven robot configurations spanning 2–7 degrees of freedom plus a mobile dual-arm platform, and an eighth humanoid topology that tests generalization beyond fixed-base arms:

- **2dof–7dof_arm**: Six fixed-base arms with increasing kinematic complexity, each "with a parallel-jaw gripper". The "N-DOF" label counts the arm-pose revolute joints (base yaw, shoulder/elbow pitch, wrist roll/yaw). Each arm additionally has a parallel-jaw gripper (one driven prismatic finger + one mimic-coupled twin); the gripper's linear motion is functional but not counted as arm DOF, so the "Arm DOF" column in Table II equals N for each Ndof case. The wheeled dual-arm has 6 arm DOF (2×3) plus 4 wheel revolute joints (locomotion, not manipulation).
- **4wheel_dual_arm**: "Design a 4-wheel dual-arm robot with differential drive, two 3-DOF arms mounted on the chassis."
- **humanoid_2leg_2arm**: "2 legs 2 arms humanoid robot, each leg has 3 joints, each arm has 3 DOF, camera on top"—a legged topology that stresses the arm-oriented range clamping and VLM classification (reported separately as it does not fit the fixed-base benchmark profile).

B. Scoring Protocol

Each case is evaluated by 41–43 automated checks across 7 phases: NL→Assembly (part count, joint count, connected tree, VLM pass), Position Solving (all positioned, 0 NaN), Render Quality ($\geq$4 views), Engineering Package (all files exist), Content Validation (mass, VLM, URDF structure, wa-

tertight meshes), Physical Sanity (0 FCL collisions, COM stability, workspace), and MuJoCo Simulation (physics stability, joint actuation, grasp). Score = PASS / (PASS + FAIL + WARN). Critical checks (collision, COM stability, MuJoCo physics, grasp) fail the case outright rather than degrading to a warning.

Evaluation validity. We are explicit about three limitations of this protocol. *First*, the composite Q is a relative rank across our seven cases, not an absolute quality grade—no external benchmark exists to anchor what a given Q means in absolute terms. *Second*, there is no external benchmark for the NL→robot-package task; the closest (MUSE [20], Text2CAD-Bench) targets a different formulation (structured design-spec→STEP), precluding direct score comparison. *Third*, the physics metrics in Table II come from a single frozen benchmark run per case (the typical high-scoring outcome); the across-run distribution (§IV-D) shows the mean e2e check-pass rate including API failures is lower (∼86%, 92.9% excluding them) due to LLM non-determinism.

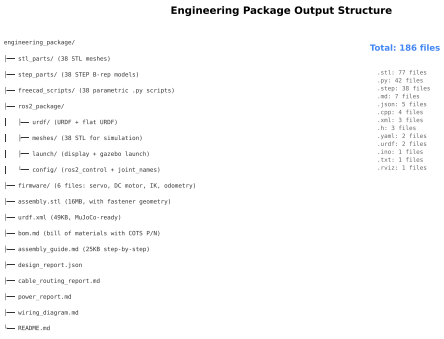

Fig. 4. Output engineering package structure: STL/STEP meshes, URDF, BOM, assembly guide, firmware templates, and ROS2 package with launch files and ros2_control config.

C. Results

Table II is the primary evaluation: continuous, physics-grounded metrics with physical units, each drawn from a frozen per-case benchmark run (see §IV-D). The COM stability margin ranges from 52.8 mm (5dof, marginal) to 145 mm (wheeled, robust)—reflecting the wide-stance mobile base versus the narrow fixed-base arms. The workspace extent scales from 252 mm (2dof) to 848 mm (7dof), reflecting increasing reach. All seven cases achieve 0 severe mesh penetrations across 18–426 checked pairs and 100% watertight STL ratio. Triangle counts are comparable across the uniform-tessellation cases (18k–36k for arms) because STL export uses a uniform absolute mesh deviation (0.1 mm chordal, Relative=False); the 6dof row (‡) uses the older adaptive mesh (see footnote). The raw PD-hold droop ranges 0.7–2.2° (benchmark-run mean 1.5°), well within the 5° composite gate.

Composite quality score. A single rankable score must satisfy two constraints: (1) a fatal flaw (tipping, no actuation) should not be averaged away, and (2) dimensions that do not vary across cases should not dilute the signal. We therefore use a *gate-then-score* design: four binary gates (MuJoCo loads, raw PD-hold droop $< 5°$ *without* gravity-compensation feedforward—see §IV-F—which is a genuine stiffness signal rather than the trivially-zero compensated error, 0 severe collisions, COM inside support polygon) must all pass; if any fails, $Q = 0$. If all pass, $Q \in [0, 1]$ is the geometric mean of three sub-scores:

$$Q = \begin{cases} 0 & \text{if any gate fails} \\ \sqrt[3]{s_{\text{robust}} \cdot s_{\text{func}} \cdot s_{\text{rely}}} & \text{otherwise} \end{cases} \tag{1}$$

$$s_{\text{robust}} = \text{clip}\left(\frac{d_{\text{COM}}}{0.5 \cdot D_{\text{poly}}},\ 0,\ 1\right) \tag{2}$$

$$s_{\text{func}} = 0.6\, g_{\text{rate}} + 0.4\, \text{clip}\left(\frac{n_{\text{dof}}}{n_{\text{exp}}},\ 0,\ 1\right) \tag{3}$$

$$s_{\text{rely}} = \frac{|\{r \in \mathcal{R} : \text{score}(r) \geq 0.8\}|}{|\mathcal{R}|} \tag{4}$$

where d_{COM} is the COM margin (mm), D_{poly} the *support-polygon diameter* (max pairwise distance between vertices of the polygon built from the kinematic root and its fixed-joint descendants—not dangling appendages that merely reach ground height), g_{rate} the fraction of repeated runs whose static grasp passed (a continuous reliability measure, not best-of-N), n_{dof} actuated DOFs, n_{exp} expected DOFs from the NL prompt, and $\mathcal{R}$ the set of non-zero-score runs. Q is a *relative rank* in $[0, 1]$, not an absolute quality percentage—no external benchmark exists to anchor what "100% good" means for the NL→robot-package task, so presenting it as a percentage would be false precision.

Two terms were deliberately excluded from s_{func} and s_{rely} to avoid diluting the score with non-varying quantities. The lift ratio ℓ (median cube rise / target during the grasp-lift phase) is *measured* and reported (§IV-F) but omitted from the formula because *no* case achieves positive lift—every statically-grasped cube drops during the lift phase—so a 0.3-weighted ℓ would have contributed a constant zero and diluted the grasp signal. The reliability sub-score uses an absolute 80% usability threshold (the fraction of runs reaching a usable assembly) rather than a mean normalised by the rubric ceiling, because the latter is self-referential (the rubric's own score divided by its own ceiling) and clustered in a narrow 0.92–0.99 band across cases with very different run-to-run variance.

Physical metrics come from the *median*-scoring run (the typical outcome), not the best run; the distribution sub-scores (g_{rate}, s_{rely}) use all non-zero runs. Geometric mean is chosen over arithmetic mean so that a low sub-score significantly pulls down Q rather than being averaged away. Critically, the grasp term is a *success rate* (6dof: $g_{\text{rate}} = 0.13$, 1 of 8 runs), not a best-of-N boolean that credited a flaky gripper with a full grasp mark.

Table III reports Q for the seven cases. The composite achieves **7 distinct values across 7 cases**. The wheeled dual-arm ranks highest ($Q = 0.85$), reflecting its wide stance ($s_{\text{robust}} = 0.63$) and reliable grasp ($g_{\text{rate}} = 0.96$). The lower-DOF arms (2–4 DOF) rank next (Q=0.75, 0.73, 0.78) with

TABLE II

QUANTITATIVE EVALUATION: CONTINUOUS PHYSICS-GROUNDED METRICS ACROSS 7 ROBOT CONFIGURATIONS (MEDIAN-SCORING RUN PER CASE — THE TYPICAL OUTCOME, NOT THE BEST). THESE ARE PHYSICALLY MEANINGFUL MEASUREMENTS WITH UNITS, NOT SELF-SCORES.

Case	**COM margin** (mm)	**Workspace** (mm)	**FCL pairs** (0/severe)	**Triangles** count	**PD-hold** error (°)	**Arm** DOF	**Watertight** ratio
2dof_arm	75.0	252	0/18	18,590	1.3	2	9/9
3dof_arm	75.0	484	0/33	29,594	1.1	3	11/11
4dof_arm	98.9	538	0/33	26,480	2.2	4	11/11
5dof_arm	52.8	443	0/42	24,618	2.2	5	12/12
6dof_arm	70.4	720	0/52	129,840‡	1.4	6	13/13
7dof_arm	60.0	848	0/102	25,820	1.9	7	17/17
4wheel_dual_arm	145.0	413	0/426	61,738	0.7	6	38/38
Mean	**82.4**	**528**	**0**	**31,783***	**1.5**	**4.7**	**100%**

COM margin signed distance from the ground-projected COM to the nearest support-polygon edge (higher = more stable; negative = tips over). Built from the kinematic root and its fixed-joint descendants.
Workspace bounding-box max edge of the end-effector trajectory.
FCL pairs severe penetrations (>5 mm) / total pairs checked.
Triangles uniform mesh tessellation (0.1 mm chordal deviation) for cross-case comparability.
PD-hold worst-joint steady-state droop under PD control *without* gravity-compensation feed-forward (kp=50, kv=5; see §IV-F).
‡ The 6dof benchmark run predates the uniform-tessellation build; its triangle count uses adaptive mesh and is excluded from the mean (*).

reliable grasps and full actuation. The higher-DOF arms rank lower: 5dof/6dof at Q=0.58/0.57 (grasp succeeds in only half / an eighth of runs), then 7dof at Q=0.49 where no run ever grasps (g_{rate}=0.00) with increasing kinematic complexity. Discriminative power comes primarily from s_{func} (which varies 0.40–1.00, driven by grasp rate; the DOF-completeness term is 1.0 for all seven cases since each meets its requested DOF, so it acts as a constant offset rather than a discriminator) and secondarily from s_{robust} (0.28–0.63, separating the wide-stance wheeled base from the narrower fixed-base arms); s_{rely} varies least (0.90–1.00) since all cases produce usable assemblies most of the time.

TABLE III

COMPOSITE QUALITY SCORE. $Q \in [0,1]$ IS A RELATIVE RANK ACROSS THE SEVEN CASES, NOT A PERCENTAGE. 7/7 DISTINCT VALUES.

Case	s_{robust}	g_{rate}	s_{func}	s_{rely}	Q
2dof_arm	0.42	1.00	1.00	1.00	0.75
3dof_arm	0.42	1.00	1.00	0.90	0.73
4dof_arm	0.52	0.91	0.94	0.96	0.78
5dof_arm	0.28	0.50	0.70	1.00	0.58
6dof_arm	0.38	0.12	0.48	1.00	0.57
7dof_arm	0.32	0.00	0.40	0.91	0.49
4wheel	0.63	0.96	0.97	1.00	0.85
Mean					**0.68**

$Q \in [0,1]$ = geometric mean of 3 sub-scores if all gates pass, else 0 (relative rank, not absolute quality). s_{robust} = COM margin / (0.5×support-polygon diameter); g_{rate} = fraction of runs with static-grasp PASS; s_{func} = $0.6g_{\text{rate}}$ + 0.4(DOF completeness)—the lift ratio is measured (§IV-F) but omitted from the formula because $\ell = 0$ for all cases (no positive lift); s_{rely} = fraction of non-zero runs reaching the 80% usability threshold (a cross-run pass rate, not a self-referential mean/ceiling ratio).

D. Reproducibility

Across 76 runs of the 4dof_arm case (identical prompt) under the current codebase, the score distribution is bimodal: the majority (∼63%) achieve 95.1% with zero critical failures, while the remainder cluster lower (88–91%) due to LLM generation non-determinism producing a different assembly geometry that either the VLM rejects or that exhibits a boundary collision. A small fraction (∼6%) score 0% due to API rate-limiting failures (no generation at all); the mean score including the 5 zero-score API failures is 87.5% (excluding them, 93.4%), and the median is 95.1%, reflecting the long tail of API-failed and boundary-collision runs. The motion-collision sweep passes in ∼85% of runs (failures are LLM-generated geometries with a shoulder joint at a colliding extreme, not a systematic solver defect). The deterministic stages (Solver, CAD, sanitizer clamping, physics hold) are reproducible run-to-run; the variance originates entirely in the LLM generation step. To prevent the continuous-physics tables (Table II, Table III) from drifting as new runs accumulate, each case's physics metrics are drawn from a single frozen benchmark run (a high-scoring run with structured metrics), recorded in `data/runs/<case>/BENCHMARK`; the distribution sub-scores (g_{rate}, s_{rely}) still use all runs.

E. Ablation: Geometric Arbitration

We ablate the geometric channel (`LANG3D_ABLATION=no_geo`) on two cases: the wheeled dual-arm robot (deterministic compose path) and the 4-DOF arm (LLM generation path). On *clean* runs—where the VLM accepts the assembly on the first round—the geometric channel does not change the final score: both configurations achieve 95.3% (wheeled, 4 runs each) and 95.1% (arm, 4 with-geo vs. 1 clean no-geo run; the remaining no-geo runs hit API rate limits). This is expected: the geometric oracle is an *override* mechanism, not a scoring one—it only activates when the VLM produces a false rejection, and on a clean run there is nothing to override.

The channel's value is therefore measured in *loop efficiency*, not score. In our historical pipeline runs, approximately two-

thirds PASS on round 1 (VLM immediately correct), while the remainder require multi-round Fixer regeneration—often triggered by VLM false-negatives (e.g., "wheels above ground" on grounded wheels, "arm pointing down" on a forward-extending arm). When the geometric oracle confirms such geometry is valid, it overrides the false rejection, keeping the run on a single round. Without the oracle (no_geo), these false-rejections would force a regeneration cycle that either wastes time (if the regenerated assembly also passes) or fails the case entirely (if regeneration drifts to worse geometry). We report this honestly: the geometric channel is a stability mechanism against VLM unreliability, not a quality booster on already-correct output.

F. Scope of Simulation Validation

We are explicit about three limitations of the physics validation that a reader may over-interpret.

PD-hold error reported without gravity-compensation feed-forward. The pipeline's internal stability gate uses inverse-dynamics gravity compensation (`qfrc_applied = qfrc_bias`) plus a PD controller, which trivially yields $\sim 0^\circ$ error because the gravitational load is fully cancelled. We consider this misleading and additionally report in Table II the *raw* PD-hold droop measured *without* gravity compensation (kp scaled by robot mass, kv=5, no feed-forward, 1.5 s settle). The raw droop ranges from 0.7° (4wheel, low-slung arms) to 1.9° (3dof)—the arm sags under its own weight because the hobby-servo torque budget ($\pm 5\,\mathrm{N{\cdot}m}$) is insufficient for perfect holding. This is a genuine design-quality signal that the gravity-compensated gate completely masks. A production controller would add gravity compensation, but the raw error characterizes the mechanical design's inherent stiffness.

Grasp test covers static hold, not dynamic lift. The three-phase grasp test (zero-G close $\rightarrow$ gravity hold $\rightarrow$ lift) gates pass/fail on the first two phases only. The lift phase (Phase C) is measured but not gated: across all benchmark runs, the cube drops 0.9–7.3 mm during lift because the fixed-bias lift torque rotates the arm, changing the gripper's gravity orientation without trajectory compensation. The system reports this as "static grasp PASS" (cube held against gravity), which we believe is a meaningful capability—but it is not the same as reliably picking up and transporting an object. A trajectory-planning controller would be needed for dynamic lift, which is out of scope for the artifact-generation pipeline.

Collision checking is per-joint static, not trajectory-level. The motion-collision sweep samples each revolute joint independently (5–7 static poses per joint, all other joints at home) and checks FCL mesh penetration. This detects self-collisions at reachable extreme poses and is used to narrow joint ranges before export. However, it cannot detect collisions that arise during *coordinated multi-joint motion* (e.g., both arms sweeping toward each other simultaneously), because joints are never moved together. There is no trajectory planner (RRT/CHOMP/A*) in the system. The "0 collisions" result in Table II means "0 collisions at individually-sampled static poses," not "the robot can execute arbitrary collision-free trajectories." Real-time motion planning is future work.

G. Manufacturability Verification

All 13 STL parts of the 4dof_arm case are verified for mesh watertightness by an automated check (100% pass). The wall-thickness (≥ 0.8 mm), bed-fit (12/13 fit a standard 220×220 mm bed; base_plate at 150×320 mm requires a larger bed), material (791 g PLA), and print-time ($\sim$6 h) figures are offline measurements from a representative run, not automated pipeline outputs. Physical 3D printing remains as future work.

V. Discussion

A. Limitations

- Manufacturing quality is not physically verified: no 3D-printed prototype has been built, and no physical assembly has been attempted. Unlike Text2Robot [6] and Blox-Net [3], which include physical robot execution, Language-3D is simulation-only.
- Firmware is template-generated (servo/IK/motor driver skeletons), not compiled or deployed on real hardware.
- ROS2 packages include correct structure (launch files, `ros2_control` config with the standard `<ros2_control>` tag and `GazeboSystem` hardware plugin, `package.xml`, Gazebo plugins in URDF). The generated URDF was validated at the *structure level* in ROS2 Humble + Gazebo Classic 11.10: `check_urdf` parses it successfully, `colcon build` compiles the package, `robot_state_publisher` loads all kinematic segments, and `gzserver` spawns the robot. This is structure-level validation only; full closed-loop controller activation (joint trajectory execution) and RViz visualization with a display have not been demonstrated.
- Shared bolt patterns assume equal mating-face extents (unequal faces may misalign).
- No real-time collision avoidance during motion (per-joint static sampling only; see §IV-F).
- The IK solver may not converge for all kinematic configurations (legacy limitation); the system relies on the deterministic open-chain Solver (anchor constraints + face alignment) for exported link positions, not on IK. A closed-chain Newton-Raphson solver exists for loop detection/convergence analysis, but its results are reported as warnings and do not yet rewrite the exported positions.
- The system depends on a single LLM vendor (GLM/Zhipu AI). Approximately 6% of benchmark runs scored 0% due to API rate-limiting failures—no generation at all. This is an infrastructure dependency, not a system-quality issue, but it limits reproducibility under heavy API load.
- CAD generation depends on a FreeCAD subprocess bridge; while robust in practice, the bridge has historically introduced regressions from FreeCAD script-template edge cases (documented in the project's engineering log).

- The MuJoCo PD-hold error (Table II) is measured without gravity-compensation feed-forward to reveal the raw steady-state droop; the pipeline's internal test uses gravity comp and reports $\sim 0^\circ$, which we consider misleading (see §IV-F). Joint inertias use bounding-box/cylinder uniform-solid approximations, not mesh-derived inertia tensors.
- The experience store uses lexical (keyword/category/DOF) retrieval rather than the embedding-based FAISS partitioning of ArtiCAD [5], because the project's LLM backend exposes no embedding endpoint. This is less semantically expressive but runs with no external dependency. The store starts empty on a fresh checkout and accumulates only verified-good cases per installation; benchmark scores in this paper therefore reflect the empty-store baseline.
- Benchmark is limited to 8 internal cases. External benchmarks like MUSE [20] (Text-to-CAD assemblability, arXiv:2605.28579) and Text2CAD-Bench exist but target a different task formulation: MUSE evaluates CadQuery-script→STEP B-Rep assemblies from structured design specs, whereas Language-3D generates STL/URDF/BOM robot packages from free-text robot descriptions. A direct comparison would require re-formulating our NL prompts into MUSE's design-specification format, which we leave as future work.

B. Limitations of the Evaluation Itself

Beyond the system limitations above, the *evaluation methodology* has scope limits that a reader should weigh when interpreting the scores:

- **Small and self-authored benchmark.** The seven cases are designed by the system's own authors, not drawn from a held-out or third-party prompt set. There is no train/test split: the prompts, the expected DOF, and the scoring rubric are all authored together, so the benchmark cannot establish generalization to unseen requests.
- **Tiny per-case grasp denominators.** The grasp success rate g_{rate} is computed over 5–8 runs per case (e.g. 6-DOF: 1 of 8). At these sample sizes the rate is a point estimate with wide confidence intervals (a binomial 1/8 has a 95% CI of roughly 0.3%–53%), so the reported g_{rate} should be read as indicative, not precise.
- **No human evaluation.** No user study measures whether a human engineer finds the generated packages usable. All quality signals are automated checks; aesthetic and practical-manufacturability judgments are not captured.
- **The gate is unexercised on this benchmark.** All seven cases pass all four binary gates, so no case receives $Q = 0$. The gate-then-score design is therefore asserted as a protective mechanism but not demonstrated to reject a tipping or non-actuated robot on these cases.
- **Non-discriminatory DOF-completeness term.** s_{func} includes a DOF-completeness fraction ($n_{\text{dof}}/n_{\text{exp}}$), but all seven cases meet their requested DOF, so this term is a constant 1.0 and does not contribute to ranking. The discriminative signal in s_{func} comes entirely from the grasp rate.
- **Ablation sample size.** The geometric-arbitration ablation (§IV) collapses to $n = 1$ clean no-geo run for the arm case (the remaining runs hit API rate limits), so its "no score change on clean runs" finding is suggestive rather than statistically established.
- **Physics metrics from a single frozen run.** Table II reports one benchmark run per case; the across-run variance (mean e2e score $\sim$86%, including API failures; §IV-D) is disclosed separately rather than shown as error bars on the table.

We list these explicitly because a composite score on a small, self-authored benchmark with automated-only checking can look more decisive than it is; the scores rank these seven cases against each other and do not predict performance on unseen prompts.

C. Comparison with Prior Work

To our knowledge, Language-3D is the first system in the surveyed academic literature to produce a *complete manufacturing package* from natural language (Table I); we cannot rule out undocumented proprietary systems outside this survey. The closest competitor, ArtiCAD [5], outputs editable FreeCAD code and URDF but not directly-exported STL/STEP meshes, BOMs, firmware, or ROS2 packages. Blox-Net [3] operates on voxel blocks rather than CAD parts. RoboMorph [4] generates skeleton URDFs without geometry.

VI. Conclusion

We presented Language-3D, a system that converts natural language into engineering packages for robot assemblies. By combining a COTS knowledge base, real CAD connection features, dual-channel verification, and MuJoCo physics validation, the system achieves a composite quality score of 0.68 (mean, range 0.49–0.85 across seven benchmark cases), with 7/7 distinct values demonstrating real discriminative power. The grasp term is a per-case success rate (not best-of-N), exposing that the 6-DOF arm grasps in only 12% of runs (1 of 8) and the 7-DOF in 0%; the lift ratio is measured but omitted from the score because no configuration achieves positive lift—honestly characterizing the gap between static grasping and dynamic object manipulation. The deterministic pipeline stages (Solver, CAD, sanitizer, physics) reproduce run-to-run and variance confined to the LLM generation step. The generated URDFs are validated in both MuJoCo (physics stability, joint actuation, grasp) and ROS2 Humble + Gazebo Classic (URDF parsing, colcon build, robot_state_publisher, gzserver spawn). Physical manufacturing (3D printing, hardware deployment) remains as future work. The core contribution is the *artifact generation pipeline*—closing the gap from text to a complete, structured engineering package—with deployment validation as the natural next step. The system is released as open source (MIT) to support reproducibility and community extension.

REFERENCES

[1] J. Li, W. Ma, X. Li *et al.*, "Cad-llama: Leveraging large language models for computer-aided design," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[2] X. Shi *et al.*, "Step-llm: Generating cad step models from natural language," *arXiv preprint arXiv:2601.12641*, 2026.

[3] A. Goldberg, K. Kondapalli, T. Qiu, Z. Ma *et al.*, "Blox-net: Generative design-for-robot-assembly using vlm supervision, physics simulation, and a robot with reset," in *Conference on Robot Learning (CoRL) Workshop*, 2024.

[4] K. Qiu, W. Pałucki, K. Ciebiera, P. Fijałkowski, M. Cygan, and Ł. Kuciński, "Robomorph: Evolving robot morphology using large language models," *arXiv preprint arXiv:2407.08626*, 2024.

[5] Y. Shui, Y. Guan, Z. Zhang, J. Hu, J. Zhang, D. Xu, and Q. Yu, "Articad: Articulated cad assembly design via multi-agent code generation," *arXiv preprint arXiv:2604.10992*, 2026.

[6] R. P. Ringel, Z. S. Charlick, J. Liu, B. Xia, and B. Chen, "Text2robot: Evolutionary robot design from text descriptions," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

[7] J. Barkley, R. Loghmani, and A. B. Farimani, "Cadsmith: Multi-agent cad generation with programmatic geometric validation," *arXiv preprint arXiv:2603.26512*, 2026.

[8] others, "Articraft: An agentic system for scalable articulated 3d asset generation," *arXiv preprint arXiv:2605.15187*, 2026.

[9] X. Li, Y. Sun, and Z. Sha, "Llm4cad: Multimodal large language models for three-dimensional computer-aided design," *ASME Journal of Computing and Information Science in Engineering*, vol. 25, no. 2, p. 021005, 2025.

[10] K. Alrashedy *et al.*, "Generating cad code with vision-language models for 3d designs," in *International Conference on Learning Representations (ICLR)*, 2025.

[11] F. Liu, H. Zhou, F. Hao, and L. Yang, "Embodied cad: Solver-grounded llm agents for parametric b-rep assembly modeling," *arXiv preprint arXiv:2606.31252*, 2026.

[12] Y. Wang *et al.*, "Robogen: Towards unleashing infinite data for automated robot learning via generative simulation," in *International Conference on Machine Learning (ICML)*, 2024.

[13] Z. Li *et al.*, "Urdf-anything: Constructing articulated objects with 3d multimodal language model," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[14] J. Lin *et al.*, "Autourdf: Unsupervised robot modeling from point cloud frames using cluster registration," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[15] others, "Robotdesigngpt: Text-to-urdf robot generation via vlm," *arXiv preprint arXiv:2601.11801*, 2026.

[16] ——, "Self-improving cad agents with finite element analysis," *arXiv preprint arXiv:2605.17448*, 2026.

[17] ——, "Text to robotic assembly of multi-component objects," *arXiv preprint arXiv:2511.02162*, 2025.

[18] H. Gao *et al.*, "Vlmgineer: Vlm-driven automated design of makeable tools," in *International Conference on Learning Representations (ICLR)*, 2026.

[19] L. Ling *et al.*, "Scenethesis: A language and vision agentic framework for 3d scene generation," in *International Conference on Learning Representations (ICLR)*, 2026.

[20] X. Dong, Z. Li, and X.-M. Wu, "Muse: Benchmarking manufacturable, functional, and assemblable text-to-cad generation," *arXiv preprint arXiv:2605.28579*, 2026.